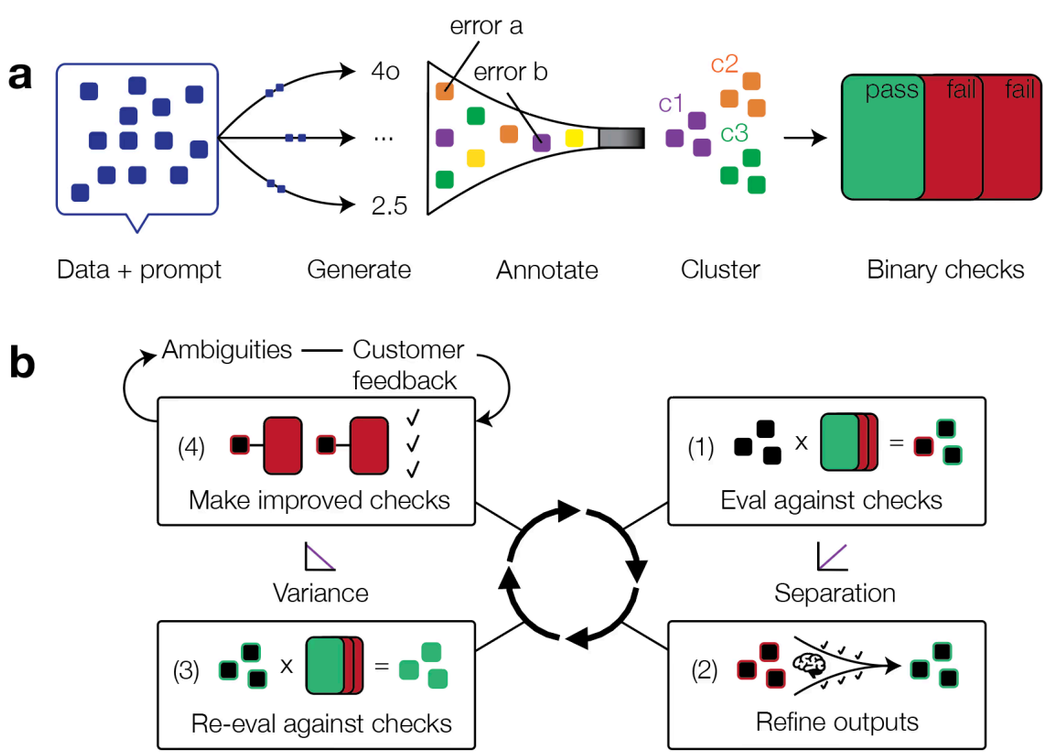
ber 30, 2025

building self-improving on through customer-in-the-inement

e evaluation engine that learns to judge like a subject matter expert.

Authors	Affiliations
Charles O'Neill	Parsed
Harry Partridge	Parsed
Max Kirkby	Parsed
Jonathon Liu	Parsed
Paras Stefanopoulos	Parsed



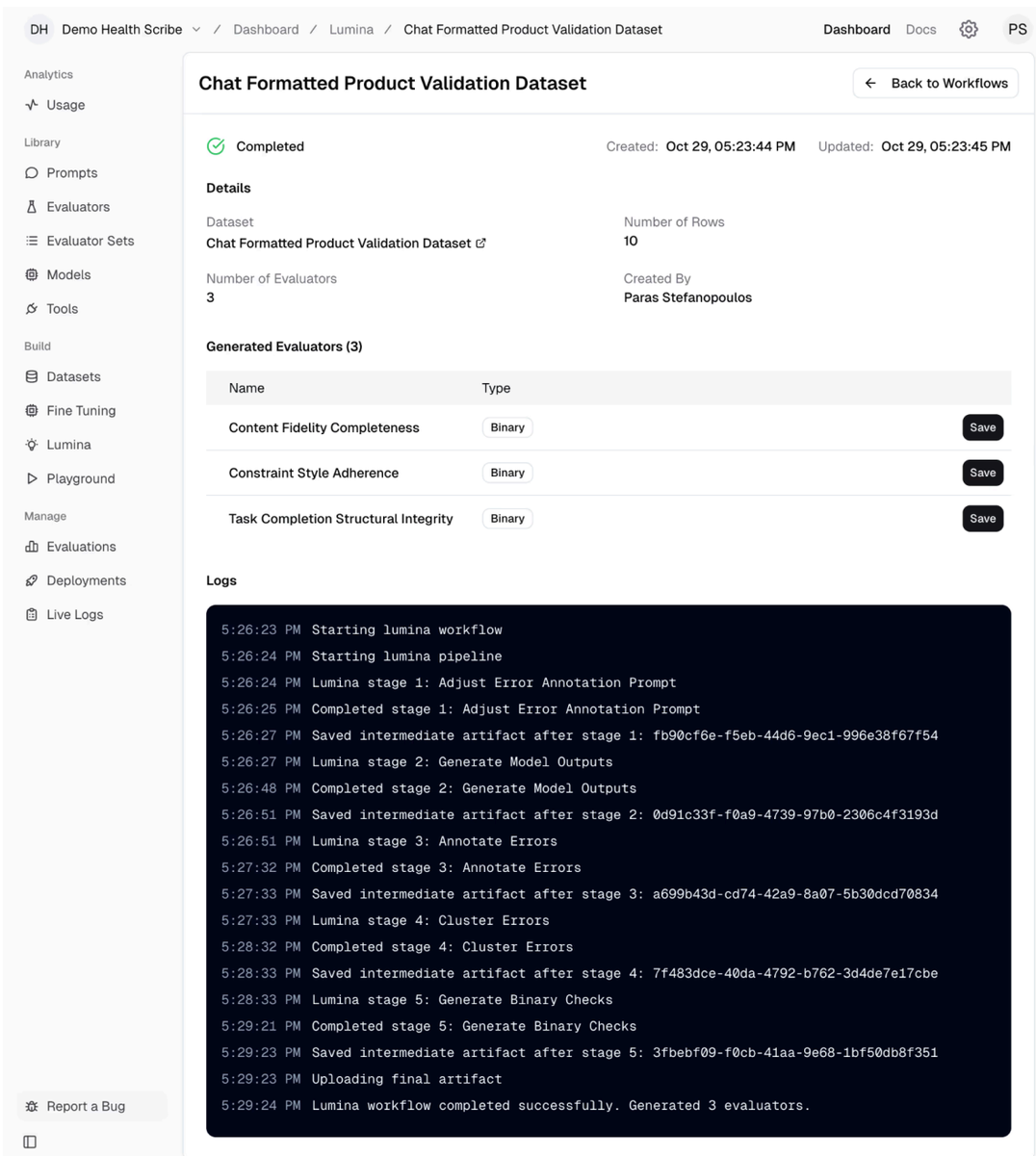
The Lumina evaluation pipeline. (a) Discovery phase: customer data and prompts generate outputs across multiple models with varying quality, which are annotated for errors, clustered into semantic categories (c1-c3), and converted into binary pass/fail checks. (b) Iterative refinement loop: evaluators are continuously improved through four stages: evaluating outputs against checks, using genetic optimisation to refine outputs for better separation, re-evaluating to measure variance reduction, and incorporating customer feedback on ambiguities to create improved checks. This cycle repeats until evaluators converge to match expert judgment.

To us, the axiom of using LLM-as-judges is that there is no free lunch. Your information on *how* to evaluate a given execution of a task by an LLM, using an LLM, needs some injection of information from the real world. Typically, this looks like extracting all the latent context and domain knowledge a customer has and objectively specifying it, in a clear, codified way, in a set of evaluators (LLM-as-judges). You need to know *what* it is the

customer would like to measure (which we've written about before), and then you need to be able to specify in precise terms *how* to measure it.

In contrast, static, out-of-the-box evaluations conceal true sources of failure. We now know what teams require is a system, a framework, by which to systematically calibrate their evaluations with the domain experts within their team who have highly opinionated and informed takes on what constitutes 'good', and how to even define that. Importantly, we also need to do so dynamically, in a way that learns from real data flowing through these systems.

To solve this problem, we built Lumina. In doing so, we've begun to treat evaluation itself as an adaptive process. One that discovers failure patterns, evolves its own tests, refines outputs and surfaces ambiguities that the only true expert, the customer, can resolve. Lumina is an evaluation system by which we can accurately and truthfully separate good and bad model outputs, taking in customer data and iteratively "touching grass" with the customer until we have a set of LLM-as-judge evaluators that collectively score any given execution of task.



Lumina running in our platform. Upload a dataset of inputs to your LLM, select the number of evaluators, and let it cook.

Lumina is really cool because it gives us a super reliable signal to hillclimb. It's what allows us to warm start the process of getting an open-source model to outperform the closed generalist models, through either RL (using the set of LLM-as-judges as reward models), SFT (using evaluators to filter out good data to train on a la rejection sampling), or our own (much better) methods like iSFT and RGT.

Establishing something to iterate on

Lumina starts with a generic error-annotation prompt (a sort of meta-prompt) after which we run a spectrum of models over as much input (often tens of thousands of examples) as we can responsibly evaluate. We run both base models and state-of-the-art reasoning models, persisting our generated outputs. This gives us a large spectrum of "quality" to meta-evaluate.

From there, we let the data shape the evaluation itself, using an annotation model to detect errors in the generated outputs. Basically, we instruct a really good model (usually Opus 4.1) to look at the generation prompt instructions, any additional context provided by the customer, and the outputs themselves, and ask: where did this fall short? In what way did it fall short? What's not perfect about this particular output?

We then cluster those errors hierarchically into a taxonomy that spans all observed failures. Rather than acting as labels, these clusters are concrete and executable checks. Lumina turns each cluster into a binary check, allowing an LLM-as-judge to evaluate an extremely precise question and record a structured PASS or FAIL with clear rationale.

We term this process 'discovery', providing artifacts for downstream iteration:

1. Take generic error-annotation prompt.
2. Generate outputs with all models.
3. Annotate outputs.
4. Cluster.
5. Create binary evaluation criteria from observed failures.

What do the evaluators (and evaluator sets) actually look like?

Each evaluator examines a single dimension of quality through a specific binary question—for instance, "does this medical note use correct tooth notation throughout?" or "are all drug dosages within safe prescribing limits?" Rather than asking for ambiguous 1-10 scores where LLMs struggle with calibration, we provide detailed context on how to objectively make each binary decision. The evaluator outputs both a structured `rationale` explaining its reasoning and a PASS/FAIL `score`.

The power comes from aggregation: a set of evaluators targeting different failure modes collectively forms a comprehensive assessment. Their binary outputs sum to create a quasi-Likert score providing both granular feedback (which checks failed with rationales) and an overall metric (18/25 passed). This modular architecture lets us add, remove, or refine individual evaluators without rebuilding the entire framework; each check evolves independently while contributing to the holistic assessment.

Importantly, this is just a starting point. This partitions the error space into semantically related checks that, taken together, should holistically assess the quality of an individual output. However, they're nowhere near perfect yet. Again, you need to inject information, and this has to come from somewhere external, the real world, touching grass.

Evaluation is nothing without its iteration: customer-in-the-loop refinement

These checks seed an iterative loop: we evaluate, refine, evaluate again, and use failures to improve the evaluator itself. In doing so, we aim to ensure that (1) a given check makes the same decision when asked again (low variance), (2) that it cleanly distinguishes acceptable from unacceptable behaviour (strong separation), but most importantly, it (3) agrees with the customer.

To extract the customer's latent knowledge about the task, Lumina discovers "ambiguities", or edge cases, that directly generate questions to be asked to the customer. For instance,

if a prompt says “list only confirmed diseases, make no inference” should a listing of “basal insulin and glucose-monitoring supplies” allow the model to infer that the patient may have diabetes? Only a customer can properly provide a true evaluation of these edge cases. We’re often surprised by how many implicit thresholds and subjective decision points are inherent in a task. We automatically send batches of these ambiguities, and their questions are automatically fed back into the pipeline to continue to refine the evaluations.

But what about the invisible gaps that might exist between evaluation criteria and the context of the task? Here we use a genetic optimization process to, at each stage (version controlled), generate the optimal output (for every input) with the current evaluation framework. As the customer annotates how closely this output matches ‘perfection’, we are able to continually steer the evaluation criteria toward optimality. If an example is perfect according to the current evaluators, but a customer can spot problems with it, that’s a really good, explicit signal we can use to update the evaluators.

Eventually, we converge to a set of evaluators which represent fairly faithfully how an expert from the company would evaluate a given output for a task.

Measuring the quality of evaluations

During iteration, to reduce variance, we measure repeat-judge consistency by re-scoring the exact same input/output pairs multiple times and, where useful, across different evaluator models. We aggregate the results of replicate checks across runs, tracking whether prompt revisions truly reduce variance over time or rather just reshuffle errors.

For separation, we examine the distribution of judgments for “good” versus “bad” examples, monitoring whether these targeted refinements (and thereby prompt versions) increase the fraction of clear PASS/FAIL decisions. In a similar vein, we also examine the consistency of FAIL rationales within a check to understand whether they are pointing at the same underlying criterion. Both of the above signals are continuously fed back into the pipeline to adjust the evaluators.

To complement our methods to reduce variance and increase separation, we also embed static analysis. We analyze evaluation prompts for things such as missing constraints relative to the generation task (when looking at evaluators as a whole), and analyze generation prompts for conflicting instructions with individual evaluators, among other things. It is ultimately these findings that are folded into prompt synthesis alongside our failure evidence, allowing our evaluations to gain specificity.

Finally, we measure agreement with the domain-experts from the company through a bunch of different metrics. What’s cool is that we can feed in arbitrary information about agreement/disagreement (the disagreement itself, the annotation from the customer about why, and along what axis, etc) in order to refine the evaluators further.

Our tracking and improvement of these ‘meta signals’ iteratively improves each stage of our criteria.

What does this mean for training?

The Lumina framework generates meaningful and consistent evaluations that can be repeatedly used for benchmarking model performance. But the important thing is that this can be used in training.

For a defined set of inputs, we are able to run genetic optimization (with Lumina as fitness) to generate ‘gold standard’ outputs that comprehensively satisfy all identified evaluation criteria. Using these golden outputs, only made possible with Lumina, we are thus able to SFT an open-source model to achieve (with increasing data quantity) the frontier level performance of models such as Gemini-2.5-Pro or Claude Opus 4.1.

This does not only apply to SFT: logically, we have observed that RLing open-source models with Lumina itself as a reward function can achieve similar performance, although it's often much slower as you're throwing away a lot of explicit supervisory signal by just relying on the score rather than the rationale as well, which tells you precisely what went wrong or right. We've written a lot about how we achieve much more sample efficient hillclimbing on these evals here and here.

This pipeline of course has consequences for continually improving performance. As more new examples are observed and evaluated, further edge cases, error categories and ambiguities can be found and eliminated. And using LLM-as-judge as the learning signal is just the start (it's our way to get around the cold-start problem); we eventually integrate all sorts of learning signals that are a bit noisier but closer to the end-user's preferences: scalar feedback, edits, acceptance rates, unstructured text feedback, etc.

Conclusion

Lumina discovers failures, converts them into executable checks and iteratively improves itself with customer judgement. In creating Lumina, we have created an evaluation system that yields benchmarks that are stable, separable and most importantly actionable in being able to steer our model and judge training. The result is not just better measurement but objectively better models. Rather than training on often messy, general customer data, Lumina allows us to train on gold-standard outputs defined by the customer themselves. Of course, it's not perfect yet, and it's also not completely automated, but Lumina has allowed Parsed to build ground truths into the models we provide.

Future work

Our immediate roadmap focuses on moving Lumina from batch-based refinement to continuous online learning. Rather than periodic evaluation cycles, we're implementing infrastructure to run Lumina continuously on production data streams, allowing evaluators to adapt in real-time as new failure modes emerge in deployment. This shift (rolling out to customers shortly) fundamentally changes the feedback loop from weeks to hours. Beyond online adaptation, we're exploring how to explicitly optimise evaluators for discriminative power: not just measuring quality accurately, but maximising the separation between models that genuinely understand a task versus those that merely pattern-match. This involves developing evaluation criteria that expose capability gaps invisible to standard benchmarks. Finally, we're training specialised judge models for each component of the Lumina pipeline (error annotation, ambiguity detection, criteria synthesis) rather than relying on general-purpose models. These purpose-built judges will bring both efficiency gains and improved accuracy, as models trained specifically for meta-evaluation tasks consistently outperform generalist models prompted for the purpose. Together, these directions point toward a future where evaluation isn't just a measurement tool but an active learning system that continuously improves both itself and the models it judges.

Other research.

BYO SWE-grep: automatically train blazing fast search sub-agents on your knowledge base (Pt. 1)

RL-trained search subagents that learn your knowledge base's structure for fast, reliable retrieval

Research Nov 11, 2025

Purpose-built LLMs for dental note-taking

Frontier thinking model performance at a fraction of the latency.

Case study Nov 5, 2025

Upweight the strategy, not the tokens: faster training with explicit reasoning through RGT (Rationale-Guided Training)

Teach the why, not just the what: Rationale-Guided Training

Research Oct 28, 2025

Attention-based attribution: what your model is actually looking at

Cosine similarity is cosplay. Attention is attribution.

Research Oct 28, 2025

Robust, sample efficient SFT with prompt mutations

Low-KL divergence prompt mutations: better performance at a fraction of the cost.

Research Oct 27, 2025

Training loss predicts evaluation performance, even for non-verifiable tasks

Loss: the cheapest evaluation you'll ever run.

Research Oct 27, 2025

Building production AI for regulated industries with a leading digital insurer

From frontier OpenAI/Google models to open-source — delivering 8x the speed and outperforming GPT-5-level accuracy.

Case study Oct 20, 2025

Iterative SFT (iSFT): dense reward learning

Iterative SFT: dense, high-bandwidth learning

Research Oct 15, 2025

Write small, learn forever: rank-1 LoRA for continual learning

Why rank-1 LoRA updates might be the missing link between static fine-tuning and truly continuous, live-on-GPU learning.

Research Oct 12, 2025

Practical LoRA Research

Fine-tuning at Scale: What LoRA Gets Right (and LoRA-XS Doesn't).

Research Oct 10, 2025

A letter to the C-suite: the shifting role of MLEs

Your MLEs are brilliant, but you're giving them the wrong job.

Position Sep 8, 2025

Fine-tuning small open-source LLMs to outperform large closed-source models by 60% on specialized tasks

27B open-source model outperforms biggest OpenAI/Anthropic/Google models on real healthcare task.

Case study Aug 15, 2025

Amnesiac generalist behemoths are not the future of language models

You don't need a generic genius. You need a specialist learner.

Position Jul 28, 2025

The bitter lesson of LLM evals

Turning expert judgment into a compounding moat. Because in LLM evals, scaling care beats scaling compute.

Position Jul 13, 2025

Do transformers notice their own mistakes? Finding a linear hallucination detector inside LLMs

A linear signal in LLMs reveals hallucinations, is detected by a frozen observer, and steered with a single vector.

Research May 8, 2025

Resurrecting the salmon: seeing clearer inside LLMs with domain-specific SAEs

A powerful, efficient, and domain-robust strategy for safeguarding medical-text generation.

Research Feb 15, 2025

Why mechanistic interpretability needs a paradigm inversion

The conventional scaling paradigm for language models themselves may be fundamentally misaligned with interp.

Research Jan 13, 2025

Start owning your model today.

From training to deployment, we help you launch a specialist LLM that outperforms generic models, adapts automatically, and runs reliably at scale.

Get started >

© 2025 Parsed Systems Inc. All rights reserved.